

Kennesaw State University

CS 3502 Operating Systems

Project 1 - System call

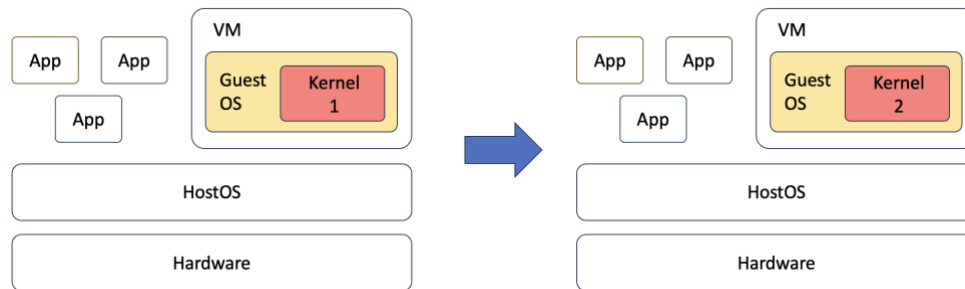
Instructor: Kun Suo

Points Possible: 100

Difficulty: ★★☆☆☆

Part A: Build the Linux kernel (30 points)

--- **Please work this part on the VMs you built on your own laptop**



Create a virtual machine using VirtualBox or UTM (for Apple Silicon) on your laptop. As the kernel compiling is pretty large, please make sure your VM has at least 4GB memory and 80GB storage (if your disk is small, create a 60GB disk for the VM). For the Operating System, use Ubuntu 22.04 iso: <https://releases.ubuntu.com/jammy/>. There exist images for two architectures: x86 (e.g., Intel, AMD) and arm (e.g., Apple silicon). Please select the one that fits your machine.

How to build one Ubuntu VM?

Windows 10 (x86):

<https://www.youtube.com/watch?v=QbmRXJJKsvs>

MacOS (x86):

<https://www.youtube.com/watch?v=GDoCrfPma2k&t=321s>

MacOS (arm):

<https://youtu.be/O19mv1pe76M?si=4cYayFiqPNoHoY1w>

Step 1: Get the Linux kernel code

Before you download and compile the Linux kernel source, make sure you have development tools installed on your system. We recommend you work this project on your virtual machine.

In Ubuntu, install this software using apt:

```
$ sudo apt-get install -y gcc libncurses5-dev make wget flex bison vim libssl-dev libelf-dev
```

To obtain the version of your current kernel, type:

```
$ uname -r  
5.15
```

For newer distributions of Ubuntu, you can see 5.x or 6.x. Please screenshot it and save it as the original kernel version.

Here is the screenshot of my current kernel:



```
ksuo@ksuo-vm ~/linux-5.19> uname -r  
5.15.0-79-generic
```

Then, download kernel 5.19 and extract the source:

```
$ wget https://cdn.kernel.org/pub/linux/kernel/v5.x/linux-5.19.tar.gz  
$ tar xvf linux-5.19.tar.gz
```

We will refer LINUX_SOURCE to the top directory of the kernel source. Go to the linux source code folder:

```
$ cd linux-5.19
```

Step 2: Configure your new kernel

Before compiling the new kernel, a .config file needs to be generated in the top directory of the kernel source. To generate the config file and make possible changes to the default kernel configurations, type:

```
$ make localmodconfig
```

For Y/N, select all "N" if any questions on the terminal to minimize the configuration file. All other selection, just select the default one. (There could be tens or even hundreds Y/N to choose, just be patient, here is sample on my side:

<https://youtu.be/hnROoXITvcc?si=br2g1xb8iGESvx-S>)

Here, we avoid using `$ make menuconfig` to save the kernel compiling time. You can check .config using the following command under kernel folder.

(<https://youtu.be/UyOGF4UOoR0>)

```
$ ls -al
```

Here is a screenshot of my VM:

```
ksuo@ksuo-vm ~/linux-5.19> ls -al
total 1660
drwxrwxr-x 24 ksuo ksuo 4096 Aug 24 20:50 .
drwxr-x--- 17 ksuo ksuo 4096 Aug 24 20:48 ..
drwxrwxr-x 24 ksuo ksuo 4096 Jul 31 2022 arch
drwxrwxr-x 3 ksuo ksuo 12288 Aug 24 20:51 block
drwxrwxr-x 2 ksuo ksuo 4096 Aug 24 20:48 certs
-rw-rw-r-- 1 ksuo ksuo 20322 Jul 31 2022 .clang-format
-rw-rw-r-- 1 ksuo ksuo 59 Jul 31 2022 .cocciconfig
-rw-rw-r-- 1 ksuo ksuo 205832 Aug 24 20:48 .config
-rw-rw-r-- 1 ksuo ksuo 372907 Aug 24 20:39 .config.old
-rw-rw-r-- 1 ksuo ksuo 496 Jul 31 2022 COPYING
-rw-rw-r-- 1 ksuo ksuo 101376 Jul 31 2022 CREDITS
```

-----[Possible Error]-----

For some distributions of Ubuntu, you may see errors like this when compiling:

No rule to make target 'debian/canonical-certs.pem', needed by 'certs/x509_certificate_list'.

[Solution]

Edit the .config file and change the value of CONFIG_SYSTEM_TRUSTED_KEYS to null

\$ vim .config

Before:

CONFIG_SYSTEM_TRUSTED_KEYRING=y

CONFIG_SYSTEM_TRUSTED_KEYS="debian/canonical-certs.pem"

After:

CONFIG_SYSTEM_TRUSTED_KEYRING=y

CONFIG_SYSTEM_TRUSTED_KEYS=""

If the CONFIG_SYSTEM_REVOCATION_KEYS="debian/canonical-revoked-certs.pem" is not null, please also set it as null as:

CONFIG_SYSTEM_REVOCATION_KEYS=""

Then recompile the kernel using the make command.

Step 3: Compile the kernel

Please keep in mind that the compiling might take 0.5-1 hour, depending on your machine hardware specs and speed. For instance, in my 2021 Macbook, it takes about 20 mins.

In LINUX_SOURCE, compile to create a compressed kernel image:

\$ make

If your VM has more than 1 core, we suggest you use "make -j N" to accelerate the compiling. Here, N denotes the number of CPUs on your VM.

To compile kernel modules:

```
$ make modules
```

You can use "make modules -j N" to accelerate the compiling. Here N denotes the number of CPUs on your VM.

Step 4: Install the kernel

Install kernel modules (become a root user, use the su command):

```
$ sudo make modules_install
```

Install the kernel:

```
$ sudo make install
```

If you are using Ubuntu, you need to create an init ramdisk manually:

```
$ sudo mkinitramfs -o /boot/initrd.img-5.19.0
```

```
$ sudo update-initramfs -c -k 5.19.0
```

The kernel image and other related files have been installed into the /boot directory. You can check it from /boot/grub/grub.cfg. Linux will boot by default using the 1st menu item.

Step 5: Modify grub configuration file

If you are using Ubuntu: change the grub configuration file:

```
$ sudo vim /etc/default/grub
```

Make the following changes:

```
GRUB_DEFAULT=0
```

```
GRUB_TIMEOUT=10
```

If your GRUB_HIDDEN_TIMEOUT_QUIET=true, change it to GRUB_HIDDEN_TIMEOUT_QUIET=false. If there is no GRUB_HIDDEN_TIMEOUT_QUIET, just ignore it.

If your GRUB_TIMEOUT_STYLE=hidden, change it to GRUB_TIMEOUT_STYLE=menu. If there is no GRUB_TIMEOUT_STYLE, just ignore it.

Then, update the grub entry:

```
$ sudo update-grub2
```

Step 6: Reboot your VM

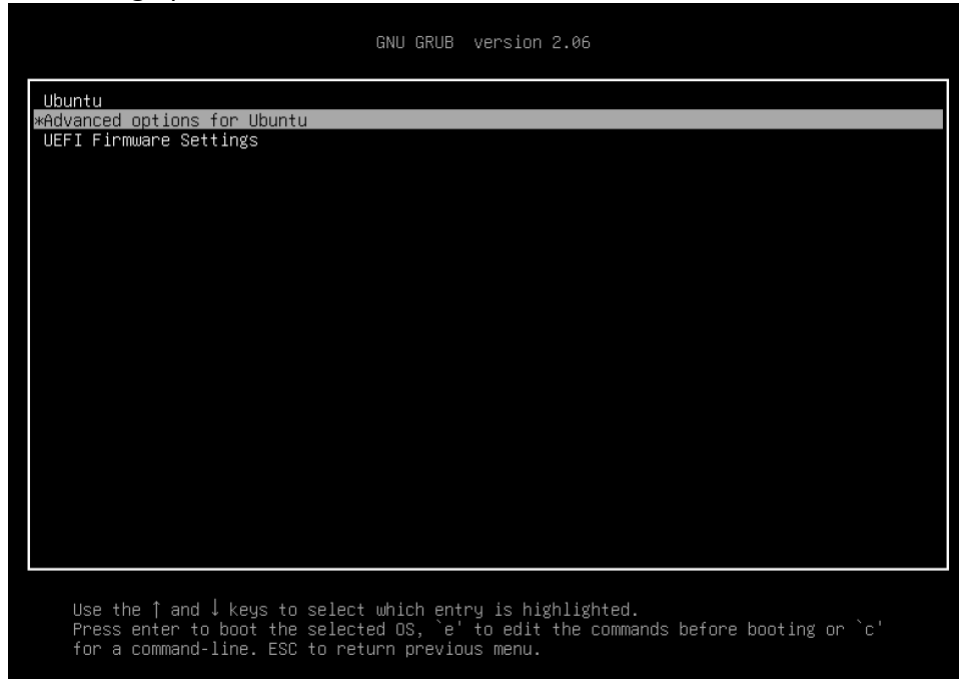
Reboot to the new kernel:

```
$ sudo reboot
```

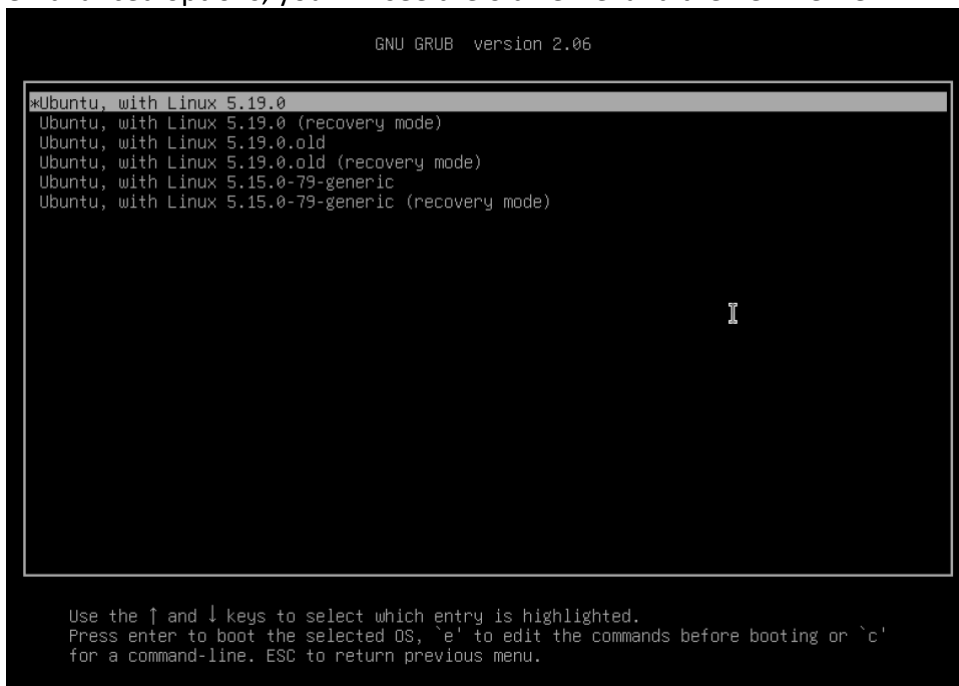
(If you are using university VM, ignore the following steps. It only works for local VM)

Immediately after the BIOS/UEFI splash screen during boot, with BIOS, quickly press and hold the Shift key, which will bring up the GNU GRUB menu. (If you see the Ubuntu logo, you've missed the point where you can enter the GRUB menu.)

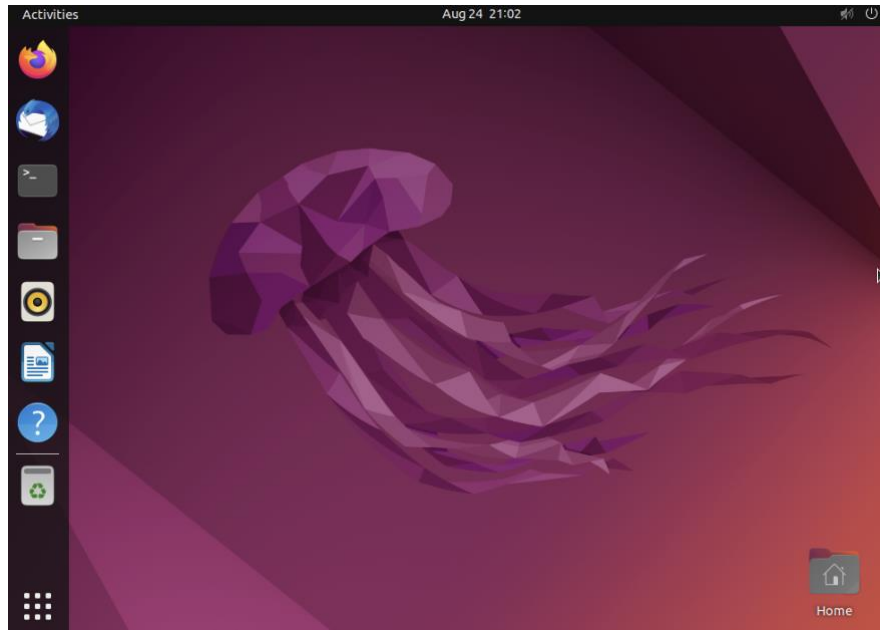
Select the following option:



Under the Advanced options, you will see the old kernel and the new kernel:



Select the new kernel and wait for a few seconds, you will enter the VM with new kernel:

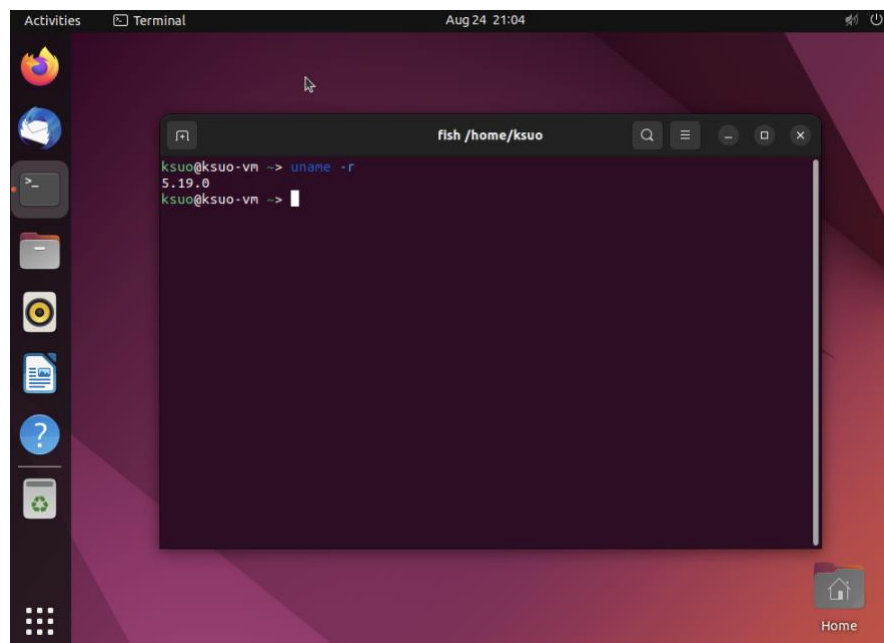


After boot, check if you have the new kernel:

```
$ uname -r
```

```
5.19.0
```

Here is the screenshot of my new kernel:

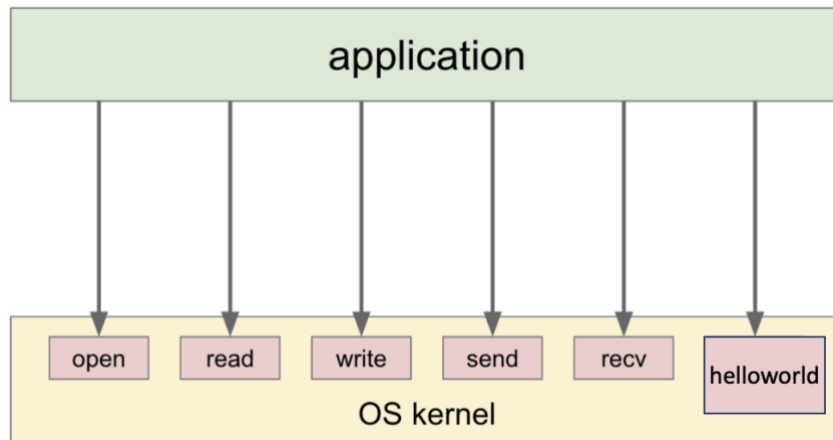


(⚠) Submission of Part A:

Please submit a report with the screenshot of `$uname -r` in your old and new kernel.

Part B: Add a new system call into the Linux kernel (40 points)

--- Please work this part on VM in Part A (kernel version = 5.19)



In this assignment, we add a simple system call `helloworld` to the Linux kernel. The system call prints out a hello world message to the syslog. You need to implement the system call in the kernel and write a user-level program to test your new system call.

If your system is x86 architecture, the following files are needed to modify:

File	Change
kernel/helloworld/helloworld.c	New — syscall implementation
kernel/helloworld/Makefile	New — build file
arch/x86/entry/syscalls/syscall_64.tbl	Added syscall entry
include/linux/syscalls.h	Added declaration
kernel/Makefile	Added helloworld/ to obj-y

Step 1: Add the System Call Implementation

Under Linux source code folder, create a new file for your syscall:

```
$ mkdir -p kernel/helloworld
$ cat > kernel/helloworld/helloworld.c << 'EOF'
#include <linux/kernel.h>
#include <linux/syscalls.h>

SYSCALL_DEFINE0(helloworld)
{
    printk(KERN_INFO "Hello, World! from syscall\n");
    return 0;
}
EOF
```

Create a Makefile for it:

```
$ cat > kernel/helloworld/Makefile << 'EOF'
obj-y := helloworld.o
EOF
```

Step 2: Register the Syscall in the Syscall Table

Edit the x86 syscall table:

```
$ nano arch/x86/entry/syscalls/syscall_64.tbl
```

Add your entry at the end of the common section. Find the last entry (around 450+) and add the following:

```
451 common helloworld sys_helloworld
```

Step 3: Declare the Syscall in the Header

```
$ nano include/linux/syscalls.h
```

Find the end of the file (before the last #endif) and add:

```
asmlinkage long sys_helloworld(void);
```

Step 4: Hook Into the Build System

Edit kernel/Makefile to include your new directory:

```
$ nano kernel/Makefile
```

Find the line starting with obj-y = (it's a long multi-line list). Add your directory at the end of that block:

```
obj-y += helloworld/
```

Repeat step 3 and 4 in PART A to re-compile, install the kernel and reboot to the new kernel.

Step 5: write a user-level program to test your system call

Go to your home directory and create a test program. Compile it and test the new system call by running.

```
$ cat > test_hello.c << 'EOF'
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>

#define SYS_helloworld 451 /* use the number you assigned */

int main() {
    long ret = syscall(SYS_helloworld);
```

```

    printf("syscall returned: %ld\n", ret);
    return 0;
}
EOF

```

```

$ gcc -o test_hello test_hello.c
$ ./test_hello

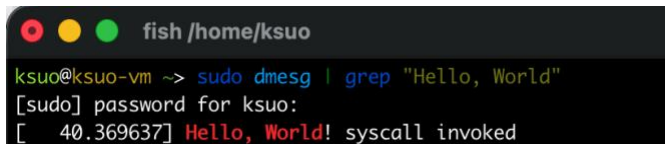
```

Then check the kernel log:

```

$ sudo dmesg | grep "Hello, World"
# Expected: [ ... ] Hello, World! from syscall

```



```

fish /home/ksuo
ksuo@ksuo-vm ~-> sudo dmesg | grep "Hello, World"
[sudo] password for ksuo:
[ 40.369637] Hello, World! syscall invoked

```

If your system is arm64 architecture (like Macbook Apple silicon), the following files are needed to modify:

File	Change
include/uapi/asm-generic/unistd.h	Add syscall number 451
include/linux/syscalls.h	Add prototype
kernel/helloworld.c	Implement the syscall
kernel/Makefile	Add helloworld.o to build

Step 1: Define the System Call Number

Edit the ARM64 syscall table:

```

$ vim arch/arm64/include/asm/unistd32.h
# for 32-bit compat (optional)

```

For 64-bit, open:

```

$ vim include/uapi/asm-generic/unistd.h

```

Find the last syscall entry (around `__NR_syscalls`) and add:

```

#define __NR_helloworld 451
__SYSCALL(__NR_helloworld, sys_helloworld)

```

Then update the count:

```

#undef __NR_syscalls
#define __NR_syscalls 452

```

Step 2: Add the ARM64-Specific Syscall Table Entry

```
$ vim arch/arm64/include/asm/unistd.h
```

Make sure this line exists (it usually does):

```
#define __NR_compat_syscalls 452
```

Step 3: Declare the System Call Prototype

```
$ vim include/linux/syscalls.h
```

Add near the end (before #endif):

```
asmlinkage long sys_helloworld(void);
```

Step 4: Implement the System Call

Create a new file or add to an existing one. The cleanest approach is a new file:

```
$ vim kernel/helloworld.c
```

```
#include <linux/kernel.h>
```

```
#include <linux/syscalls.h>
```

```
SYSCALL_DEFINE0(helloworld)
```

```
{
```

```
    printk(KERN_INFO "Hello, World! syscall invoked\n");
```

```
    return 0;
```

```
}
```

Step 5: Register the File in the Build System

```
$ vim kernel/Makefile
```

Find the obj-y line and add your file:

```
obj-y += helloworld.o
```

Repeat step 3 and 4 in PART A to re-compile, install the kernel and reboot to the new kernel.

Step 6: Test the System Call

Write a small C test program:

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/syscall.h>
```

```
#define __NR_helloworld 451
```

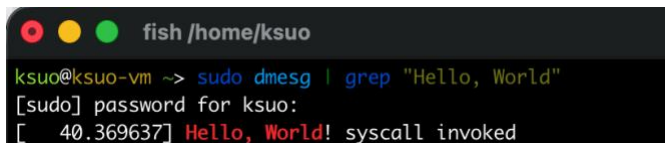
```
int main() {  
    long ret = syscall(__NR_helloworld);  
    printf("syscall returned: %ld\n", ret);  
    return 0;  
}
```

Compile and run:

```
$ gcc -o test_hello test_hello.c  
./test_hello
```

Check the kernel log:

```
$ sudo dmesg | grep "Hello, World"
```



```
fish /home/ksuo  
ksuo@ksuo-vm ~-> sudo dmesg | grep "Hello, World"  
[sudo] password for ksuo:  
[ 40.369637] Hello, World! syscall invoked
```

(⚠) Submission of Part B:

Please update the system call above so that your system call message will print out “Here is my syscall in the OS kernel by [Your Name]!”. Then, submit a report with the screenshot of \$ dmesg | grep “by Your name”.

For instance, a student named Sisi should upload a screenshot like:



```
fish /home/administrator/project1  
administrator@CS3502-14 ~/project1> dmesg | grep "by Sisi"  
[178964.009107] Here is my syscall in OS kernel by Sisi!
```

replace " " with your name

Part C: Extend your new system call to print out the calling process's information (40 points)

Follow the instructions we discussed above and implement another system call print_self. This system call identifies the calling process at the user-level and print out various information of the process.

Implement the `print_self` system call and print out the following information of the calling process:

- Process id, running state, and program name
- Start time and virtual runtime
- Its parent processes until init (first system process)

HINT: The macro ***current*** returns a pointer to the `task_struct` of the current running process. The virtual runtime (`vruntime`) is located at the `sched_entity` of `task_struct`.

`task_struct` data structure in the kernel:

<https://elixir.bootlin.com/linux/v5.1/source/include/linux/sched.h#L585>

Expected output:

```
-zsh
Start time: 19904sshd
This is task2.
Task PID: 2846
Task State: 1
Virtual runtime: 24923576313
Start time: 19898sshd
This is task2.
Task PID: 1708
Task State: 1
Virtual runtime: 37512239536
Start time: 18init
This is task2.
Task PID: 1
Task State: 1
Virtual runtime: 15488035001
Start time: 0
[suokun@ksuo-CentOS assignment2]$
```

(⚠) Submission of Part C:

Submit a report with the screenshot of the above output from your `print_self` system call.

(⚠) Submission format:

Please submit the following two files:

1. One report file containing:
 - a. All screenshots from Parts A, B, and C
 - b. A detailed discussion of your code logic and implementation
2. One ZIP file containing:
 - a. All code files from Parts A, B, and C. The Code files should be organized clearly and logically

Important: Submissions that do not follow the format above will not be graded.